

Technical Assessment and Security Analysis Report

Author: Moe Htet Ar Kar (Phoe Cho)

1. Python Development & Network Scanning

Scripting & Automation

Developed a Python-based network scanning project in Visual Studio Code, implementing asynchronous scanning modules, including `async_tcp.py`, and custom TCP/SYN scanning logic through `syn_scan.py` to support efficient network reconnaissance.

Network Reconnaissance

Executed the scanner through the terminal to perform targeted reconnaissance against specified IP addresses and identify accessible TCP ports.

2. FastAPI & Vulnerable Web API Application

API Architecture & Discovery

Analyzed the architecture of the **Mini Juice Shop Lab**, a vulnerable web application built with FastAPI. Leveraged the integrated Swagger UI endpoint (`/docs`) to inspect the application's API specification, available routes, request parameters, and response structures.

Endpoint Enumeration

Enumerated and interacted with core application endpoints associated with:

- Authentication
 - User profiles
 - User reviews
 - Scoreboards
 - Other exposed API functionality
-

3. IDOR (Insecure Direct Object Reference) Exploitation

Vulnerability Discovery

Intercepted HTTP requests using Burp Suite and modified object reference identifiers within the `/api/users/profile/{id}` endpoint.

The assessment demonstrated that the backend failed to enforce appropriate object-level authorization checks, allowing access to resources associated with another user.

Impact Validation

The vulnerability was successfully validated by accessing administrative account data and retrieving the associated lab flag:

```
FLAG{idor_profile_access_success}
```

The finding demonstrates how insufficient server-side authorization controls can result in unauthorized access to other users' resources and may enable both horizontal and vertical privilege escalation, depending on the application's authorization model.

4. Cross-Site Scripting (XSS) & Insecure Output Handling

Payload Testing

Tested the application's review functionality by submitting executable `<script>` payloads through the reviews API endpoint.

The objective was to assess whether user-controlled input was properly validated, sanitized, and contextually encoded before being rendered by the application.

Security Observation

Insufficient output handling can allow attacker-controlled content to be interpreted as executable client-side code, potentially resulting in Cross-Site Scripting (XSS).

Appropriate input validation, context-aware output encoding, and Content Security Policy (CSP) controls should be implemented to reduce the risk.

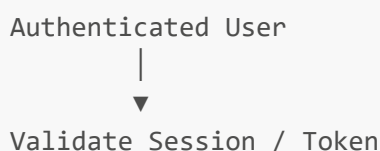
5. Recommended Security Control for IDOR

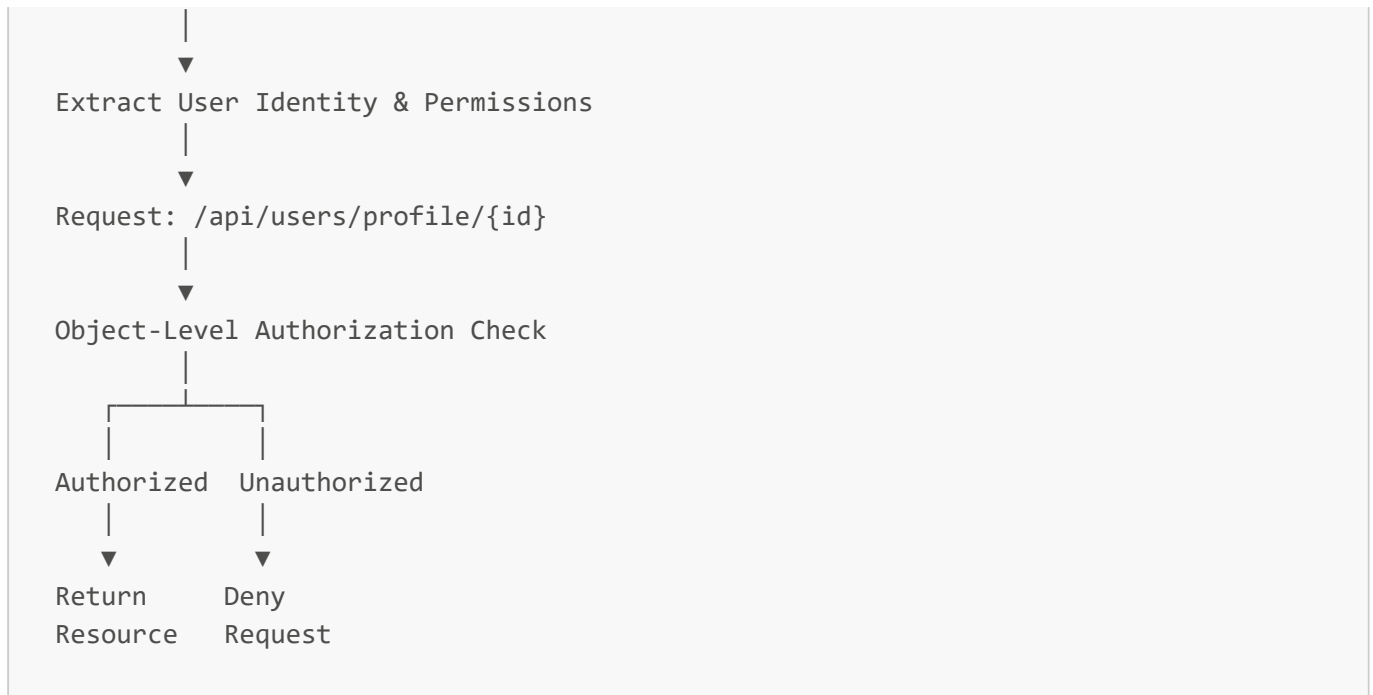
To protect the `/api/users/profile/{id}` endpoint against IDOR, the backend should implement **server-side object-level authorization checks** for every resource request.

The authorization layer should:

1. Authenticate the requesting user.
2. Identify the authenticated user's identity and privileges from the server-validated session or access token.
3. Determine whether the requested `{id}` belongs to the authenticated user or whether the user has an explicitly authorized role or permission to access that resource.
4. Deny the request if the authorization check fails.
5. Perform the authorization check on the server for every request rather than relying on client-side restrictions.

For example, conceptually:





Recommended Mechanism

The appropriate security mechanism is **server-side object-level authorization**, commonly implemented through **RBAC (Role-Based Access Control)** and/or **ABAC (Attribute-Based Access Control)** depending on the application's authorization model.

For user-owned resources, the backend should enforce an ownership check equivalent to:

```
requested_resource.owner_id == authenticated_user.id
```

For privileged users, the authorization policy may additionally verify whether the authenticated user possesses an explicit permission to access the requested object.

Security Principle

Never trust the client-supplied object identifier as proof of authorization.

The `{id}` parameter identifies the requested resource; it must not determine whether the requester is allowed to access that resource. Authorization must always be enforced server-side.